



INDIA.RUNS

Build what next India runs on

Team Name : **The Last Commit**

Team Leader Name : **Tripti Kashyap**

Problem Statement : **The Data & AI Challenge : Intelligent Candidate Discovery & Ranking**

Solution Overview

- **We reconstruct the recruiter's decision function, not another keyword matcher.**

A consensus-gated, fraud-hardened, explainable ranking engine that reads what candidates actually did (career history), not what they claim (skill lists). It ranks all 100,000 candidates in about 12 seconds, on CPU, fully offline.

Why it beats traditional matching

- **Consensus gate, not a weighted sum.** Strong keywords can never rescue a wrong role; fraud can spike one signal but cannot forge agreement across all of them.
- **Evidence from the work.** Fit is read from career descriptions, with a semantic layer that catches great engineers who never use buzzwords.
- **Integrity built in.** Impossible profiles are hard-gated; behavioral signals down-weight unavailable candidates.
- **Trustable output.** Every pick ships a fact-grounded reason plus one honest concern.

JD Understanding & Candidate Evaluation

- **Key requirements extracted from the JD (encoded as a machine-readable rubric)**

- **Must-haves:** production embedding/retrieval systems, vector DB or hybrid search operations, strong Python, ranking-evaluation rigor (NDCG, MRR, A/B).
- **Ideal band:** 6 to 8 years (5 to 9 acceptable); product-company work; in or open to Pune, Noida and Tier-1 hubs.
- **Disqualifiers:** research-only careers, LangChain-only experience, services-only careers, title-chasers, CV/speech-primary without NLP/IR.

Candidate signals that decide relevance, in order of weight

- **Role consensus:** current and past titles vs the intelligence-layer role.
- **Evidence of work:** retrieval, ranking and recsys work described in career history (keyword plus semantic).
- **Skill trust:** proficiency x endorsements x months used, grounded against the work.
- **Availability:** response rate, recency, open-to-work, notice period. A perfect profile that will not reply is not a hire.
- **Integrity:** internal consistency of dates, tenures and skill claims.

Ranking Methodology

- **Two-stage pipeline: broad evidence scoring over 100K, then consensus gating at the top.**

1. **Evidence extraction.** Career descriptions scored against the JD rubric (lexical) and five JD facet embeddings (MiniLM, precomputed offline; rank time is a scalar lookup).
2. **Consensus gate.** $\text{core} = \sqrt{\text{role_fit} \times \text{evidence_fit}}$: a geometric gate, so either pillar failing collapses the score. This is what kills keyword-stuffers.
3. **Refinements.** Skill-trust, experience band and trajectory (product vs services, tenure) refine ordering inside the gate.
4. **Multipliers.** Availability, location fit and integrity apply as bounded multipliers; honeypots are hard-gated to the bottom.
5. **Deterministic ties.** Equal scores break by candidate_id, per spec.

Signals combine multiplicatively where trust is at stake, additively where quality refines. No single forged signal can dominate.

Explainability & Data Validation

- **Explainability: grounded, falsifiable, rank-consistent**

- Every candidate gets a 1-2 sentence reason citing only real profile fields, generated deterministically from the data. Hallucination is structurally impossible.
- Each reason includes one honest concern (the weakest real signal), so tone always matches rank.

Data validation: built for a pool that lies

- **Honeypots (~80 impossible profiles)**: role duration longer than time-since-start, expert skills with 0 months used, tenure far beyond stated experience. Hard-gated; 0 in our top-100.
- **Keyword-stuffers (3,301 in pool)**: perfect AI skill lists on non-technical roles. They die at the consensus gate; 0 in our top-100.
- **Noisy inconsistencies (9% of pool)**: soft-discounted only. Hard-filtering would false-negative thousands of real people.

End-to-End Workflow

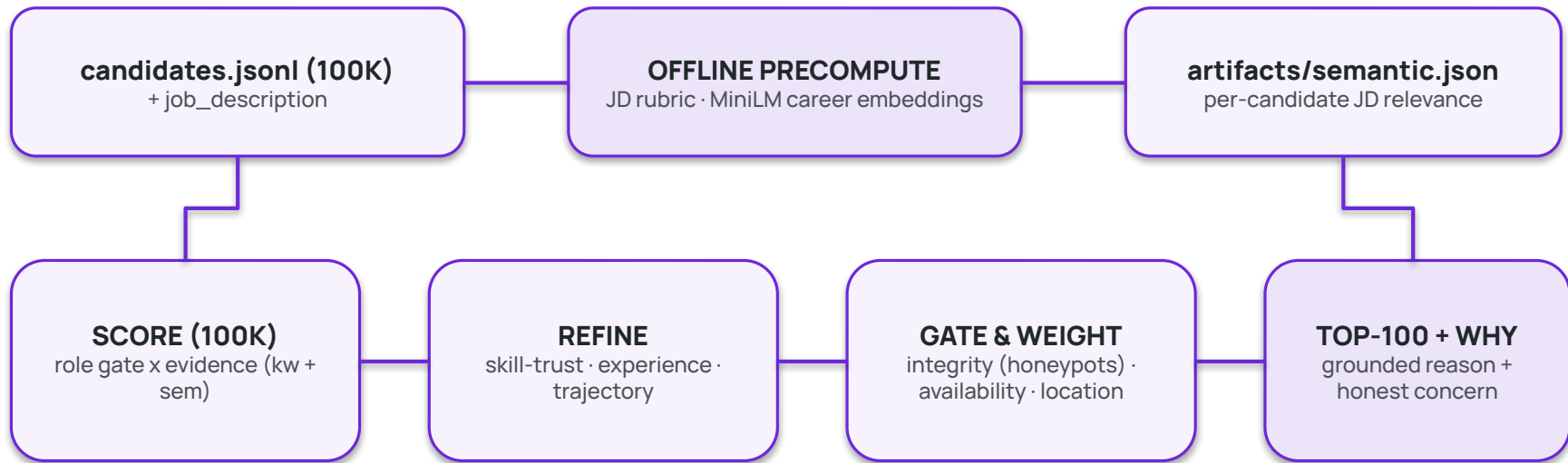
- **Offline, once:** JD to structured rubric · 100K career texts to MiniLM embeddings to per-candidate semantic scores (artifacts/semantic.json)

Rank time, about 12 seconds, CPU, no network

1. Stream candidates.jsonl (100K profiles)
2. Score: role gate x evidence (keyword plus semantic) x skill-trust x experience x trajectory
3. Apply availability, location and integrity multipliers; hard-gate honeypots
4. Sort; break ties by candidate_id
5. Re-fetch top-100 and generate grounded reasoning plus an honest concern per candidate
6. Emit CSV (candidate_id, rank, score, reasoning). Passes the official validator.

The same pipeline powers our sandbox: upload any sample of up to 100 candidates, get a ranked, explained shortlist in seconds.

System Architecture



Rank time: ~12 s for 100,000 candidates · CPU-only · zero network · under 200 MB RAM (budget: 5 min / 16 GB)

Results & Performance

0.95

composite on hand-labeled gold set

0 / 0 / 0

honeypots / stuffers / CV-primary in
our top-100

12 s

for 100,000 candidates, CPU, offline
(budget 5 min)

- Gold-set evaluation, 55 hand-labeled candidates, the challenge's own composite:

NDCG@10 0.93 NDCG@50 0.99 MAP 0.96 P@10 1.00

Full-pool trap-leak test, traps admitted into the top-100 of 100,000:

Ours: 0 honeypots · 0 stuffers · 0 CV-primary

Naive skill-keyword ranker (the JD's stated trap): 2 honeypots · 33 stuffers · 5 CV-primary

Typical title + skill% + YoE weighted sum: 2 honeypots · 0 stuffers · 10 CV-primary

Removing any single defense still leaks zero: defense-in-depth. A trap must beat the role gate, the evidence reader, skill-grounding and integrity at the same time.

Technologies Used

- **Deliberately lean. Every choice serves the 5-minute / CPU / no-network constraint.**
- Python 3.11 + orjson: streams 100K profiles (465 MB) in seconds with tiny memory.
- sentence-transformers (MiniLM-L6): offline precompute only; semantic relevance ships as a 2.3 MB artifact, so rank time needs no model at all.
- **Pure-stdlib scoring engine:** the rubric, consensus gate, integrity checks and reasoning are transparent Python. Inspectable, deterministic, interview-defensible.
- **Evaluation:** hand-labeled gold set, NDCG/MAP/P@k harness, full-pool leak tests, ablations.
- **Streamlit + Google Colab:** sandbox running the exact pipeline on any sample of up to 100 candidates.

What we deliberately did NOT use: LLM API calls at rank time (banned and unscalable), GPU dependence, and black-box learned rankers that cannot explain a single decision.

Submission Assets

GitHub repository (public)

github.com/anshul23102/redrob-candidate-ranker

Sandbox: full pipeline, one click, Run all (Google Colab)

colab.research.google.com/github/anshul23102/redrob-candidate-ranker/blob/main/notebooks/sandbox.ipynb

Reproduce our submission (CPU, offline, under 5 minutes)

```
python rank.py --candidates ./data/candidates.jsonl --out ./the_last_commit.csv
```

Ranked output: top-100 CSV with per-candidate grounded reasoning. Passes the official validate_submission.py. The repo includes the eval harness, leak tests, methodology document, submission metadata and full iterative git history.



INDIA.RUNS

Build what next India runs on

THANK YOU